

Βελτιστοποίηση Λειτουργίας Συστήματος BESS με Χρήση Python

Μαθηματικός Προγραμματισμός & Ανάλυση Arbitrage για
Φ/Β Σταθμό 5 MWp στην Αγορά Επόμενης Ημέρας

Κωνσταντίνος Ξηρογιάννης

Σεπτέμβριος 2026

DISCLAIMER

Η παρούσα εργασία αποτελεί προσωπικό project εφαρμοσμένης έρευνας και εξοικείωσης με εργαλεία ενεργειακής ανάλυσης και προγραμματισμού (Python, PuLP, ENTSO-E Data). Σε καμία περίπτωση δεν υποκαθιστά επαγγελματική οικονομοτεχνική μελέτη.

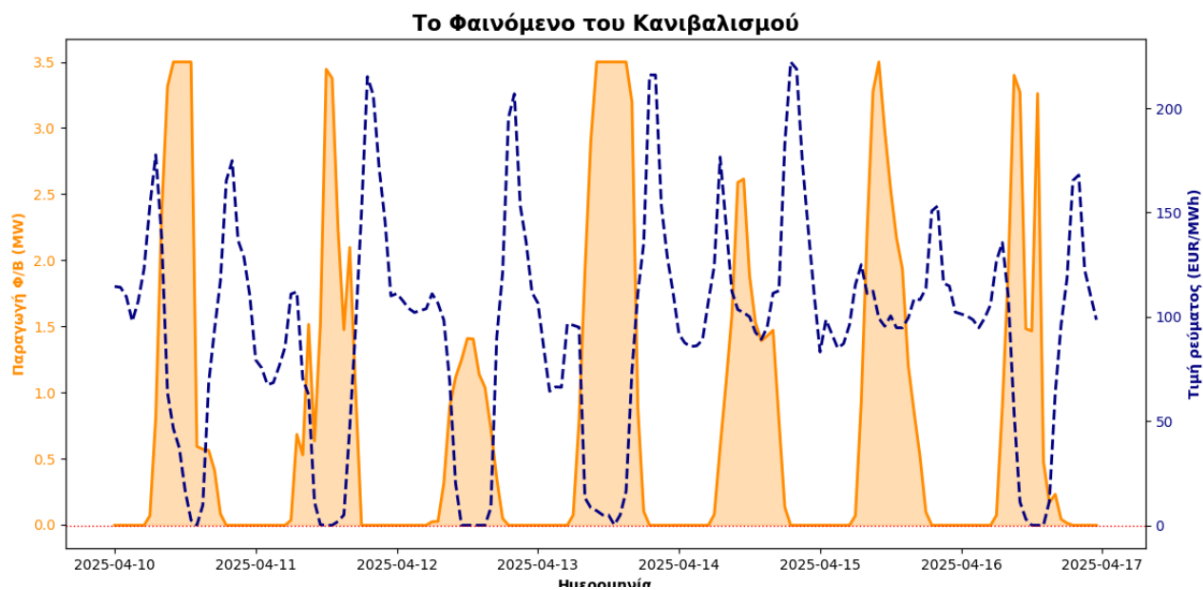
Abstract

Η παρούσα εργασία εξετάζει το φαινόμενο του «κανιβαλισμού» των τιμών ηλεκτρικής ενέργειας (Renewable Cannibalization Effect) που προκαλείται από τη μαζική διείσδυση των φωτοβολταϊκών στην Ελλάδα. Χρησιμοποιώντας πραγματικά ωριαία δεδομένα της Αγοράς Επόμενης Ημέρας (DAM) του ENTSO-E για το 2025 και δεδομένα παραγωγής από φωτοβολταϊκό πάρκο 5 MWp (**σχεδιασμένο στο PVsyst**), αναπτύσσεται προγραμματιστικό πλαίσιο σε Python για τη βέλτιστη διαχείριση συστήματος αποθήκευσης BESS (3.84 MWh / 2 MW). Συγκρίνονται ευρετικοί αλγόριθμοι (στατικά και δυναμικά όρια) έναντι ακριβούς Μαθηματικής Βελτιστοποίησης με Γραμμικό Προγραμματισμό (Linear Programming/ Simplex μέσω της βιβλιοθήκης PuLP). Τα αποτελέσματα αποδεικνύουν ότι ο Γραμμικός Προγραμματισμός επιτυγχάνει αύξηση των ετήσιων εσόδων κατά +45.9% συγκριτικά με το σενάριο του σταθμού χωρίς σύστημα αποθήκευσης, αναδεικνύοντας την κρίσιμη αξία των Energy Analytics στη σύγχρονη αγορά.

1 Εισαγωγή & Το Φαινόμενο του Κανιβαλισμού των ΑΠΕ

Η ραγδαία αύξηση της εγκατεστημένης ισχύος φωτοβολταϊκών σταθμών στην Ελλάδα και την Ευρώπη έχει μεταβάλει ριζικά τη διαμόρφωση των τιμών στη χονδρεμπορική αγορά ηλεκτρικής ενέργειας. Τις ημέρες με έντονη ηλιοφάνεια, η ταυτόχρονη υπερπροσφορά ενέργειας από τα φωτοβολταϊκά πάρκα υπερκαλύπτει τη ζήτηση του συστήματος. Αυτό έχει ως αποτέλεσμα τη δημιουργία της χαρακτηριστικής **«Καμπύλης Πάπιας» (Duck Curve)**, όπου τις μεσημεριανές ώρες η Οριακή Τιμή του Συστήματος κατακρημνίζεται σε μηδενικά ή ακόμη και αρνητικά επίπεδα.

Το φαινόμενο αυτό, γνωστό ως **«Κανιβαλισμός των ΑΠΕ» (Cannibalization Effect)**, μειώνει δραστικά την αξία της παραγόμενης ενέργειας για τους παραγωγούς. Η προσθήκη Συστημάτων Αποθήκευσης Ηλεκτρικής Ενέργειας με μπαταρίες (**BESS**) προσφέρει την απαραίτητη ευελιξία για τη χρονική μετατόπιση της ενέργειας (**Energy Arbitrage**). Η μπαταρία, δηλαδή, φορτίζει κατά τις μεσημεριανές ώρες μηδενικών τιμών και εκφορτίζει κατά τις βραδινές ώρες αιχμής, όπου η τιμή εκτοξεύεται λόγω της ένταξης θερμικών μονάδων φυσικού αερίου.



Σχήμα 1: Απεικόνιση του φαινομένου του "κανιβαλισμού" των τιμών και της παραγωγής από Φ/Β κατά την ανοιξιιάτικη εβδομάδα 10-16 Απριλίου του 2025.

2 Προέλευση & Προετοιμασία Δεδομένων

Για την ανάλυση αξιοποιήθηκαν δύο ανεξάρτητα πρωτογενή datasets ωριαίας ανάλυσης για ένα πλήρες έτος (8.760 ώρες).

2.1 Δεδομένα Παραγωγής Φ/Β)

Τα δεδομένα παραγωγής προέρχονται από την αναλυτική προσομοίωση επίγειου σταθμού **5 MWp** στην Καρδίτσα (**σχεδιασμένο σε προηγούμενο στάδιο στο PVsyst**) με τα εξής τεχνικά χαρακτηριστικά:

- **Εγκατεστημένη ισχύς:** 5.028 kWp DC / 3.850 kWac
- **Όριο έγχυσης δικτύου:** 3.50 MW
- **Χωρητικότητα BESS:** 3.84 MWh ωφέλιμη (80% DOD), μέγιστη ισχύς $P_{\text{bess}} = 2.0 \text{ MW}$
- **Ιδιοκατανάλωση βοηθητικών συστημάτων μπαταρίας/κλιματισμού:** $P_{\text{aux}} = 3.85 \text{ kW}$

2.2 Δεδομένα Τιμών Αγοράς

Εξήχθησαν οι τιμές της Αγοράς Επόμενης Ημέρας (**Day-Ahead Market - DAM**) της ελληνικής ζώνης προσφορών (BZN|GR) για το έτος 2025. Τα δεδομένα 15λεπτης ανάλυσης υποβλήθηκαν σε ομαδοποίηση ανά ώρα (*hourly resampling*) μέσω της βιβλιοθήκης **Pandas**.

	Solar_MW	Price_EUR_MWh
Datetime		
2025-01-01 00:00:00	-0.00385	138.7000
2025-01-01 01:00:00	-0.00385	134.0600
2025-01-01 02:00:00	-0.00385	124.4200
2025-01-01 03:00:00	-0.00385	118.6000
2025-01-01 04:00:00	-0.00385	112.3800
...	...	...
2025-12-31 19:00:00	-0.00385	121.0000
2025-12-31 20:00:00	-0.00385	113.6675
2025-12-31 21:00:00	-0.00385	111.2000
2025-12-31 22:00:00	-0.00385	108.0025
2025-12-31 23:00:00	-0.00385	89.7275

8760 rows × 2 columns

Σχήμα 2: Δομή του ενιαίου DataFrame

3 Στρατηγικές Διαχείρισης BESS & Ευρετικοί Αλγόριθμοι

Για τη διαχείριση της φόρτισης και εκφόρτισης της μπαταρίας υπό το επιβεβλημένο όριο έγχυσης στο δίκτυο στα **3.50 MW**, αναπτύχθηκαν αρχικά δύο ευρετικές (*heuristic*) μέθοδοι:

3.1 Στατικός Αλγόριθμος Κατωφλίου (Static Threshold)

Η μπαταρία λειτουργεί με βάση σταθερά απόλυτα όρια τιμών καθ' όλη τη διάρκεια του έτους:

$$\text{Κατάσταση}(t) = \begin{cases} \text{Φόρτιση,} & \text{εάν } \text{Price}(t) \leq 40 \text{ €/MWh} \quad \wedge \quad \text{SoC}(t) < E_{\max} \\ \text{Εκφόρτιση,} & \text{εάν } \text{Price}(t) \geq 120 \text{ €/MWh} \quad \wedge \quad \text{SoC}(t) > 0 \\ \text{Αναμονή,} & \text{διαφορετικά} \end{cases} \quad (1)$$

Η καθαρή ισχύς έγχυσης στο δίκτυο περιορίζεται δυναμικά στο όριο των 3.50 MW ($P_{\text{grid}} = \min(3.5, P_{\text{net}})$).

3.2 Δυναμικός Ευρετικός Αλγόριθμος (Smart Daily Heuristic)

Ο αλγόριθμος προσαρμόζεται καθημερινά στην καμπύλη τιμών της DAM. Για κάθε ημέρα:

1. Εντοπίζει τις **2 ώρες με τις χαμηλότερες τιμές** και δίνει εντολή φόρτισης από την περίσσεια της ηλιακής παραγωγής.

$$P_{\text{ch}}(t) = \min(P_{\text{bess, max}}, 3.84 - \text{SoC}(t), P_{\text{solar}}(t)) \quad (2)$$

2. Εντοπίζει τις **2 ώρες με τις υψηλότερες τιμές** και δίνει εντολή εκφόρτισης, λαμβάνοντας υπόψη το διαθέσιμο περιθώριο του δικτύου:

$$P_{\text{grid, avail}}(t) = \max(0, 3.5 - P_{\text{solar}}(t)) \quad (3)$$

$$P_{\text{dis}}(t) = \min(P_{\text{bess, max}}, \text{SoC}(t), P_{\text{grid, avail}}(t)) \quad (4)$$

 ΟΙΚΟΝΟΜΙΚΗ ΑΝΑΛΥΣΗ (PV 5MWp | BESS 3.84 MWh)

-
1. Απλό Πάρκο (Χωρίς Μπαταρία): 453,580 €
 2. Μπαταρία με Στατικό Αλγόριθμο (40€-120€): 547,111 € (+ 93,531 € έξτρα)
 3. Μπαταρία με Δυναμικό Αλγόριθμο (Smart): 615,960 € (+ 162,380 € έξτρα)
-

 Αξία Ανάλυσης Δεδομένων (Διαφορά Στατικού-Δυναμικού): +68,849 € !!!

Σχήμα 3: Αποτελέσματα Ευρετικών Μεθόδων

4 Μαθηματική Βελτιστοποίηση με Γραμμικό Προγραμματισμό (LP)

Παρά την αποτελεσματικότητα του δυναμικού ευρετικού αλγορίθμου, οι απόλυτοι κανόνες (if/else) αδυνατούν να εγγυηθούν το απόλυτο μαθηματικό βέλτιστο όταν οι φθηνές ώρες έπονται των ακριβών ή όταν υπάρχει περιθώριο για πολλαπλούς μερικούς κύκλους φόρτισης / εκφόρτισης της μπαταρίας. Για την εύρεση της καθολικά βέλτιστης λύσης διατυπώθηκε πρόβλημα **Γραμμικού Προγραμματισμού (Linear Programming)** και επιλύθηκε με τον αλγόριθμο Simplex μέσω της βιβλιοθήκης PuLP.

4.1 Μαθηματική Διατύπωση (LP Formulation)

Σύνολο Χρονικών Βημάτων: $\mathcal{T} = \{0, 1, \dots, 8759\}$ (ωριαία βήματα έτους).

Μεταβλητές Απόφασης $\forall t \in \mathcal{T}$:

- $P_{\text{ch}}(t) \geq 0$: Ισχύς φόρτισης της μπαταρίας [MW].
- $P_{\text{dis}}(t) \geq 0$: Ισχύς εκφόρτισης της μπαταρίας [MW].
- $P_{\text{curt}}(t) \geq 0$: Ισχύς περικοπής (curtailment) από τους inverters λόγω ορίου δικτύου [MW].
- $\text{SoC}(t) \geq 0$: Ενεργειακό περιεχόμενο (State of Charge) της μπαταρίας στο τέλος της ώρας t [MWh].

Αντικειμενική Συνάρτηση (Μεγιστοποίηση Ετήσιων Εσόδων):

$$\max_{P_{\text{ch}}, P_{\text{dis}}, P_{\text{curt}}, \text{SoC}} \sum_{t=0}^{8759} [P_{\text{solar}}(t) - P_{\text{curt}}(t) - P_{\text{ch}}(t) + P_{\text{dis}}(t) - P_{\text{aux}}] \cdot \text{Price}_{\text{DAM}}(t) \quad (5)$$

Περιορισμοί Συστήματος ($\forall t \in \mathcal{T}$):

- **Όρια Ισχύος Φόρτισης / Εκφόρτισης & Χωρητικότητας:**

$$0 \leq P_{\text{ch}}(t) \leq 2.0 \text{ MW}, \quad 0 \leq P_{\text{dis}}(t) \leq 2.0 \text{ MW}, \quad 0 \leq \text{SoC}(t) \leq 3.84 \text{ MWh} \quad (6)$$

- **Ισοζύγιο Ηλιακής Παραγωγής (Πράσινη Φόρτιση):**

$$P_{\text{ch}}(t) + P_{\text{curt}}(t) \leq P_{\text{solar}}(t) \quad (7)$$

- **Όριο Έγχυσης στο Δίκτυο:**

$$P_{\text{solar}}(t) - P_{\text{curt}}(t) - P_{\text{ch}}(t) + P_{\text{dis}}(t) - P_{\text{aux}} \leq 3.50 \text{ MW} \quad (8)$$

• **Δυναμική Εξέλιξη Κατάστασης Φόρτισης (SoC Balance):**

$$\text{SoC}(t) = \text{SoC}(t-1) + P_{\text{ch}}(t) - P_{\text{dis}}(t), \quad \forall t \geq 1 \quad (9)$$

$$\text{SoC}(0) = P_{\text{ch}}(0) - P_{\text{dis}}(0) \quad (10)$$

```
=====
Εγκαθιστούμε τη βιβλιοθήκη PuLP και ξεκινάμε τη βελτιστοποίηση
=====
Ο αλγόριθμος λύνει την εξίσωση.
```

```
 ΑΠΟΤΕΛΕΣΜΑΤΑ ΓΡΑΜΜΙΚΟΥ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ (Absolute Optimal)
```

```
=====
Κατάσταση Επίλυσης:          Optimal
1. Απλό Πάρκο (No BESS):      453,580 €
2. Μαθηματικό Μέγιστο (LP):   659,666 € (+ 206,086 € έξτρα)
=====
```

Σχήμα 4: Αποτέλεσμα Γραμμικού Προγραμματισμού

5 Συγκριτική Ανάλυση Αποτελεσμάτων & Οικονομικά Ευρήματα

Η εκτέλεση των αλγορίθμων στην Python παρήγαγε τα ακόλουθα συγκριτικά αποτελέσματα ετήσιων εσόδων:

Πίνακας 1: Συγκριτική Αποτίμηση Εσόδων & Αποδοτικότητας Αλγορίθμων

Μέθοδος / Στρατηγική	Περιγραφή Τρόπου Λειτουργίας	Έσοδα (€)	vs Base (€)
1. Base Case (No BESS)	Άμεση πώληση με όριο έγχυσης στα 3.5 MW	453.580	—
2. Στατικός Αλγόριθμος	Κατώφλια τιμών στα 40 €/MWh και στα 120 €/MWh	547.111	+93.531
3. Δυναμικός Αλγόριθμος	Έξυπνη επιλογή 2 φθηνότερων/ακριβότερων ωρών για φόρτιση/εκφόρτιση	615.960	+162.380
4. Linear Optimization (LP)	Απόλυτο Μαθηματικό Βέλτιστο (PuLP)	659.666	+206.086
Αξία Data Analytics (Δυναμικός vs Στατικός):			+68.849 €
Αξία Γραμμικού Προγραμματισμού (LP vs Δυναμικός):			+43.706 €

5.1 Αξιολόγηση Ευρημάτων

- **Ανάδειξη της Αξίας των BESS:** Η προσθήκη της μπαταρίας υπό το όριο των 3.5 MW αυξάνει τα ετήσια έσοδα κατά τουλάχιστον **+20.6%** (στατικό μοντέλο). Η βέλτιστη εκκαθάριση του συστήματος με χρήση γραμμικού προγραμματισμού αυξάνει τα ετήσια έσοδα κατά το εντυπωσιακό **+45.4%**.
- **Υπεροχή του Γραμμικού Προγραμματισμού:** Η διαφορά των 43.706 € μεταξύ της δυναμικής ευρετικής μεθόδου και του LP οφείλεται στην ικανότητα του LP να εκμεταλλεύεται ενδοημερήσιες διακυμάνσεις με πολλαπλούς μερικούς κύκλους και να αποφασίζει τη βέλτιστη κατανομή μεταξύ φόρτισης, εκφόρτισης και αναγκαίας περικοπής (curtailment).

6 Συμπεράσματα & Μελλοντικές Προεκτάσεις

Τα δεδομένα ανέδειξαν το φαινόμενο του «κανιβαλισμού» των ΑΠΕ. Το μεσημέρι οι τιμές της ηλεκτρικής ενέργειας κατακρημνίζονται φτάνοντας στο μηδέν ή παίρνοντας μέχρι και αρνητικές τιμές. Συνεπώς, ένα απλό φωτοβολταϊκό πάρκο υπόκειται σε σημαντικό οικονομικό ρίσκο. Η εγκατάσταση συστήματος BESS προσφέρει ευελιξία και καθιστά βιώσιμη την επένδυση, καθώς επιτρέπει τη μετατόπιση της ενέργειας από τις φθηνές μεσημεριανές στις ακριβές βραδινές ώρες.

Στα πλαίσια αυτά, η μελέτη απέδειξε ότι στην εποχή της υψηλής διείσδυσης των ΑΠΕ, η τεχνική αρτιότητα του εξοπλισμού (hardware) πρέπει απαραίτητα να συνοδεύεται από προηγμένο λογισμικό βελτιστοποίησης (software & analytics). Ο στατικός αλγόριθμος αύξησε το κέρδος του πάρκου κατά 93.531 €, ενώ ο δυναμικός αλγόριθμος κατά 162.380 €. Η προσαρμογή στα δεδομένα της DAM επέφερε επιπλέον έσοδα της τάξεως των 68.849 €. Τέλος, το μοντέλο της PuLP υπολόγισε το απόλυτο μαθηματικό μέγιστο κέρδος στα **659.666 €** (+206.086 € έναντι του απλού πάρκου).

Οι παραπάνω αριθμοί αποδεικνύουν πως οι ευρετικές μέθοδοι έχουν όρια. Συνεπώς, για την επίτευξη του μέγιστου οικονομικού οφέλους απαιτείται συνδυασμός Μοντέλων Πρόβλεψης Τιμών και Μαθηματικής Βελτιστοποίησης Κυλιόμενου Ορίζοντα (Model Predictive Control – MPC).

Μελλοντικές Βελτιώσεις Μοντέλου:

1. **Ενσωμάτωση Κόστους Υποβάθμισης Μπαταρίας (Battery Degradation Cost):** Προσθήκη μη γραμμικού κόστους φθοράς ανά κύκλο στην αντικειμενική συνάρτηση για αποφυγή ασύμφορων μικρο-κύκλων.
2. **Μοντέλα Πρόβλεψης Τιμών & Rolling Horizon (MPC):** Στην πράξη, οι Aggregators δεν γνωρίζουν εκ των προτέρων τις τιμές 8.760 ωρών. Απαιτείται συνδυασμός Αλγορίθμων Μηχανικής Μάθησης (Machine Learning Forecasting) και Ελέγχου Προγνωστικού Μοντέλου (Model Predictive Control – MPC) σε κυλιόμενο ορίζοντα 24–48 ωρών.

7 Πηγαίος Κώδικας Python (Google Colab)

7.1 Φόρτωση & Προετοιμασία Δεδομένων

```
1 import pandas as pd
2
3 # =====
4 # 1. ΦΟΡΤΩΣΗ ΔΕΔΟΜΕΝΩΝ
5 # =====
6 df_prices = pd.read_csv('Price per hour.csv', sep=';', decimal=',')
7 df_production = pd.read_csv('Energy per hour.csv', sep=';', decimal=',',
8     ↪ header=None)
9
10 # =====
11 # 2. ΚΑΘΑΡΙΣΜΟΣ ΠΑΡΑΓΩΓΗΣ (PVsyst)
12 # =====
13 df_production.columns = ['PVsyst_Time', 'Energy_W']
14 # Μετατροπή W σε MW
15 df_production['Energy_MW'] = df_production['Energy_W'] / 1000000
16 # Ορισμός στήλης ωρών (Datetime)
17 df_production['Datetime'] = pd.date_range(start='2025-01-01 00:00',
18     ↪ periods=8760, freq='h')
19 df_production.set_index('Datetime', inplace=True)
20
21 # =====
22 # 3. ΚΑΘΑΡΙΣΜΟΣ ΑΓΟΡΑΣ (ENTSO-E)
23 # =====
24 df_prices.columns = df_prices.columns.str.strip()
25
26 # Βρίσκουμε δυναμικά τα ονόματα των στηλών
27 time_col = df_prices.columns[0] # Η 1η στήλη (Ημερομηνία)
28 price_col = df_prices.columns[1] # Η 2η στήλη (Τιμή)
29
30 # Μετατρέπουμε την τιμή σε καθαρό αριθμό
31 df_prices[price_col] =
32     ↪ pd.to_numeric(df_prices[price_col].astype(str).str.replace(',', '.'),
33     ↪ errors='coerce')
34
35 # Φτιάχνουμε το Datetime κρατώντας τους πρώτους 19 χαρακτήρες
36 df_prices['Datetime'] = df_prices[time_col].astype(str).str.split(' -
37     ↪ ').str[0].str.strip().str[:19]
38 df_prices['Datetime'] = pd.to_datetime(df_prices['Datetime'], format='%d/%m/%Y
39     ↪ %H:%M:%S', errors='coerce')
40 df_prices.set_index('Datetime', inplace=True)
41
42 # Ομαδοποίηση δεδομένων ανά ώρα
43 df_prices_hourly = df_prices.resample('h').mean(numeric_only=True)
44
45 # =====
46 # 4. ΕΝΩΣΗ
47 # =====
48 df_final = pd.concat([df_production['Energy_MW'], df_prices_hourly[price_col]],
49     ↪ axis=1)
```

```

43 df_final.columns = ['Solar_MW', 'Price_EUR_Mwh']
44
45 print("Το τελικό, ενωμένο DataFrame (8.760 ώρες):")
46 display(df_final)

```

7.2 Οπτικοποίηση του Φαινομένου του Κανιβαλισμού των ΑΠΕ

```

1  import matplotlib.pyplot as plt
2
3  # Φιλτράρουμε τα δεδομένα για μια ανοιξιιάτικη εβδομάδα (π.χ. 10 με 16 Απριλίου)
4  df_spring = df_final.loc['2025-04-10':'2025-04-16']
5
6  # Στήνουμε τον "καμβά" του γραφήματος
7  fig, ax1 = plt.subplots(figsize=(12, 6))
8
9  # Πρώτος Άξονας (Αριστερά): Παραγωγή Φ/B (Πορτοκαλί)
10 color_solar = 'darkorange'
11 ax1.set_xlabel('Ημερομηνία', fontweight='bold')
12 ax1.set_ylabel('Παραγωγή Φ/B (MW)', color=color_solar, fontweight='bold')
13 # Γεμίζουμε με χρώμα κάτω από τη γραμμή
14 ax1.plot(df_spring.index, df_spring['Solar_MW'], color=color_solar, linewidth=2)
15 ax1.fill_between(df_spring.index, df_spring['Solar_MW'], color=color_solar,
16                 ↪ alpha=0.3)
17 ax1.tick_params(axis='y', labelcolor=color_solar)
18
19 # Δεύτερος Άξονας (Δεξιά): Τιμή DAM (Μπλε)
20 # Η εντολή twinx() φτιάχνει έναν δεύτερο άξονα y που μοιράζεται τον ίδιο άξονα x
21 ax2 = ax1.twinx()
22 color_price = 'navy'
23 ax2.set_ylabel('Τιμή ρεύματος (EUR/MWh)', color=color_price, fontweight='bold')
24 # Σχεδιάζουμε την τιμή με διακεκομμένη γραμμή για να ξεχωρίζει
25 ax2.plot(df_spring.index, df_spring['Price_EUR_Mwh'], color=color_price,
26         ↪ linewidth=2, linestyle='--')
27 ax2.tick_params(axis='y', labelcolor=color_price)
28
29 # Προσθέτουμε κόκκινη γραμμή στο μηδέν της τιμής
30 # ώστε να είναι εμφανές πότε η τιμή γίνεται αρνητική
31 ax2.axhline(0, color='red', linewidth=1, linestyle=':')
32
33 # Τίτλος και εμφάνιση
34 plt.title('Το Φαινόμενο του Κανιβαλισμού', fontsize=15, fontweight='bold')
35 fig.tight_layout()
36 plt.show()

```

7.3 Στατικός & Δυναμικός Αλγόριθμος Διαχείρισης BESS

```

1  import pandas as pd
2  import numpy as np
3
4  # =====

```

```

5 # 1. ΦΟΡΤΩΣΗ ΚΑΙ ΚΑΘΑΡΙΣΜΟΣ ΔΕΔΟΜΕΝΩΝ
6 # =====
7 # Φόρτωση τιμών
8 df_prices = pd.read_csv('Price per hour.csv', sep=';', decimal=',')
9 # Φόρτωση παραγωγής
10 df_production = pd.read_csv('Energy per hour.csv', sep=';', decimal=',',
    ↪ header=None)
11
12 # Καθαρισμός πίνακα παραγωγής
13 df_production.columns = ['PVsyst_Time', 'Energy_W']
14 df_production['Energy_MW'] = df_production['Energy_W'] / 1000000
15 df_production['Datetime'] = pd.date_range(start='2025-01-01 00:00',
    ↪ periods=8760, freq='h')
16 df_production.set_index('Datetime', inplace=True)
17
18 # Καθαρισμός πίνακα τιμών
19 df_prices.columns = df_prices.columns.str.strip()
20 time_col = df_prices.columns[0]
21 price_col = df_prices.columns[1]
22
23 df_prices[price_col] =
    ↪ pd.to_numeric(df_prices[price_col].astype(str).str.replace(',', '.'),
    ↪ errors='coerce')
24 df_prices['Datetime'] = df_prices[time_col].astype(str).str.split(' -
    ↪ ').str[0].str.strip().str[:19]
25 df_prices['Datetime'] = pd.to_datetime(df_prices['Datetime'], format='%d/%m/%Y
    ↪ %H:%M:%S', errors='coerce')
26 df_prices.set_index('Datetime', inplace=True)
27
28 df_prices_hourly = df_prices.resample('h').mean(numeric_only=True)
29 df_final = pd.concat([df_production['Energy_MW'], df_prices_hourly[price_col]],
    ↪ axis=1)
30 df_final.columns = ['Solar_MW', 'Price_EUR_Mwh']
31
32 # Καθαρίζουμε τα κενά (NaNs)
33 df_final['Solar_MW'] = df_final['Solar_MW'].fillna(0) # Αν λείπει παραγωγή -> 0
34 df_final['Price_EUR_Mwh'] = df_final['Price_EUR_Mwh'].ffill().bfill() #
    ↪ Forward/backward fill στις τιμές
35
36 # =====
37 # 2. ΠΡΟΣΟΜΟΙΩΣΗ ΜΠΑΤΑΡΙΑΣ
38 # =====
39 BESS_CAPACITY = 3.84 # MWh
40 BESS_POWER = 2.0 # MW
41 BESS_AUX_MW = 0.00385 # Η ιδιοκατανάλωση της μπαταρίας (3.85 kW) από το PVsyst
42
43 grid_static = []
44 grid_dynamic = []
45
46 prices = df_final['Price_EUR_Mwh'].values
47 solar = df_final['Solar_MW'].values
48
49 # Α. ΣΤΑΤΙΚΟΣ ΑΛΓΟΡΙΘΜΟΣ (40 EUR - 120 EUR)
50 CHARGE_PRICE = 40.0

```

```

51 DISCHARGE_PRICE = 120.0
52 battery_static = 0.0
53
54 for t in range(len(prices)):
55     p, s = prices[t], solar[t]
56     charge, discharge = 0.0, 0.0
57
58     if p < CHARGE_PRICE and battery_static < BESS_CAPACITY:
59         charge = min(BESS_POWER, BESS_CAPACITY - battery_static, s)
60         battery_static += charge
61     elif p > DISCHARGE_PRICE and battery_static > 0:
62         discharge = min(BESS_POWER, battery_static)
63         battery_static -= discharge
64
65     net_power = s - charge + discharge - BESS_AUX_MW
66     grid_static.append(min(3.5, net_power))
67
68 # B. ΔΥΝΑΜΙΚΟΣ ΑΛΓΟΡΙΘΜΟΣ
69 df_final['Date'] = df_final.index.date
70 battery_dynamic = 0.0
71
72 for date, daily_data in df_final.groupby('Date'):
73     cheapest_threshold = daily_data['Price_EUR_Mwh'].nsmallest(2).max()
74     expensive_threshold = daily_data['Price_EUR_Mwh'].nlargest(2).min()
75
76     for idx, row in daily_data.iterrows():
77         p, s = row['Price_EUR_Mwh'], row['Solar_MW']
78         charge, discharge = 0.0, 0.0
79
80         if p <= cheapest_threshold and battery_dynamic < BESS_CAPACITY:
81             charge = min(BESS_POWER, BESS_CAPACITY - battery_dynamic, s)
82             battery_dynamic += charge
83         elif p >= expensive_threshold and battery_dynamic > 0:
84             # Υπολογίζουμε πόσος χώρος υπάρχει στο δίκτυο μέχρι τα 3.5 MW
85             available_grid_capacity = max(0.0, 3.5 - s)
86             # Η εκφόρτιση περιορίζεται από την ισχύ inverters, το SoC και το
87             ↪ δίκτυο
88             discharge = min(BESS_POWER, battery_dynamic,
89                             ↪ available_grid_capacity)
90             battery_dynamic -= discharge
91
92             net_power = s - charge + discharge - BESS_AUX_MW
93             grid_dynamic.append(min(3.5, net_power))
94
95 # =====
96 # 3. ΟΙΚΟΝΟΜΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ
97 # =====
98 rev_no_bess = (df_final['Solar_MW'].clip(upper=3.5) *
99 ↪ df_final['Price_EUR_Mwh']).sum()
100 rev_static = (np.array(grid_static) * prices).sum()
101 rev_dynamic = (np.array(grid_dynamic) * prices).sum()
102
103 print("💰 ΟΙΚΟΝΟΜΙΚΗ ΑΝΑΛΥΣΗ (PV 5MWp | BESS 3.84 MWh)")
104 print("=" * 75)

```

```

102 print(f"1. Απλό Πάρκο (Χωρίς Μπαταρία): {rev_no_bess:,.0f} EUR")
103 print(f"2. Μπαταρία με Στατικό Αλγόριθμο (40EUR-120EUR): {rev_static:,.0f} EUR
    ↪ (+ {rev_static - rev_no_bess:,.0f} EUR έξτρα)")
104 print(f"3. Μπαταρία με Δυναμικό Αλγόριθμο (Smart): {rev_dynamic:,.0f} EUR (+
    ↪ {rev_dynamic - rev_no_bess:,.0f} EUR έξτρα)")
105 print("=" * 75)
106 print(f"00 Αξία Ανάλυσης Δεδομένων (Διαφορά Στατικού-Δυναμικού): +{rev_dynamic -
    ↪ rev_static:,.0f} EUR !!!")

```

7.4 Γραμμικός Προγραμματισμός με PuLP

```

1 !pip install pulp
2
3 import pulp
4 import pandas as pd
5 import numpy as np
6
7 print("=" * 60)
8 print("Εγκαθιστούμε τη βιβλιοθήκη PuLP και ξεκινάμε τη βελτιστοποίηση")
9 print("=" * 60)
10
11 # Βασικές παράμετροι
12 BESS_CAPACITY = 3.84
13 BESS_POWER = 2.0
14 BESS_AUX_MW = 0.00385
15 T = len(df_final) # 8760 ώρες
16
17 prices = df_final['Price_EUR_Mwh'].values
18 solar = df_final['Solar_MW'].values
19
20 # 1. Ορισμός Προβλήματος: Μεγιστοποίηση εσόδων (LpMaximize)
21 prob = pulp.LpProblem("BESS_Optimal_Arbitrage", pulp.LpMaximize)
22
23 # 2. Ορισμός Μεταβλητών Απόφασης
24 charge = pulp.LpVariable.dicts("Charge", range(T), lowBound=0,
    ↪ upBound=BESS_POWER)
25 discharge = pulp.LpVariable.dicts("Discharge", range(T), lowBound=0,
    ↪ upBound=BESS_POWER)
26 soc = pulp.LpVariable.dicts("SoC", range(T), lowBound=0, upBound=BESS_CAPACITY)
27 # Η μεταβλητή για το ρεύμα που "κόβει" ο Inverter εξαιτίας
28 # του ορίου έγχυσης στο δίκτυο
29 curtail = pulp.LpVariable.dicts("Curtail", range(T), lowBound=0)
30
31 # 3. Αντικειμενική Συνάρτηση: Συνολικό ετήσιο κέρδος
32 # Έσοδα = (Ήλιος - Περικοπή - Φόρτιση + Εκφόρτιση - Ιδιοκατανάλωση) * Τιμή
33 prob += pulp.lpSum([(solar[t] - curtail[t] - charge[t] + discharge[t] -
    ↪ BESS_AUX_MW) * prices[t] for t in range(T)])
34
35 # 4. Περιορισμοί
36 for t in range(T):
37     # Περιορισμός A: Η φόρτιση και η περικοπή δεν μπορούν να ξεπερνούν την
    ↪ διαθέσιμη ενέργεια από τον ήλιο

```

```

38 prob += charge[t] + curtail[t] <= solar[t], f"Solar_Balance_{t}"
39
40 # Περιορισμός B: Battery State of Charge
41 if t == 0:
42     # Την 1η ώρα του έτους: Το SoC είναι απλά Φόρτιση - Εκφόρτιση
43     prob += soc[t] == charge[t] - discharge[t], f"SoC_Balance_{t}"
44 else:
45     # Τις υπόλοιπες ώρες: Νέο SoC = Προηγούμενο SoC + Φόρτιση - Εκφόρτιση
46     prob += soc[t] == soc[t-1] + charge[t] - discharge[t],
47     ↪ f"SoC_Balance_{t}"
48
49 # Περιορισμός Γ: Grid Power Limit 3.5 MW
50 prob += (solar[t] - curtail[t] - charge[t] + discharge[t] - BESS_AUX_MW) <=
51     ↪ 3.5, f"Grid_Limit_{t}"
52
53 # 5. Λύση
54 print("Ο αλγόριθμος λύνει την εξίσωση.")
55 prob.solve()
56
57 # 6. Εξαγωγή των βέλτιστων αποτελεσμάτων
58 optimal_grid = []
59 for t in range(T):
60     c_val = charge[t].varValue
61     d_val = discharge[t].varValue
62     curt_val = curtail[t].varValue
63     optimal_grid.append(solar[t] - curt_val - c_val + d_val - BESS_AUX_MW)
64
65 rev_no_bess = (df_final['Solar_MW'].clip(upper=3.5) *
66     ↪ df_final['Price_EUR_Mwh']).sum()
67 rev_optimal = (np.array(optimal_grid) * prices).sum()
68
69 print("\n000 ΑΠΟΤΕΛΕΣΜΑΤΑ ΓΡΑΜΜΙΚΟΥ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ (Absolute Optimal)")
70 print("=" * 70)
71 print(f"Κατάσταση Επίλυσης: {pulp.LpStatus[prob.status]}")
72 print(f"1. Απλό Πάρκο (No BESS): {rev_no_bess:,.0f} €")
73 print(f"2. Μαθηματικό Μέγιστο (LP): {rev_optimal:,.0f} € (+ {rev_optimal -
74     ↪ rev_no_bess:,.0f} € έξτρα)")
75 print("=" * 70)

```